

# Multi-Table JOIN:

## The Simplest Explanation

Imagine you are building a sandwich. You start with bread, then add cheese, then add tomato, then add lettuce. Each new layer goes on top of everything you already have. You never go back to just the bare bread.

### SQL JOINS work the same way.

You start with one table. Then each LEFT JOIN adds a new layer of data on top of what you already built. The next JOIN does NOT go back to the original tables. It works on the combined result of everything so far.

| Step   | What You Add | What You Have Now                 |
|--------|--------------|-----------------------------------|
| Start  | Bread (base) | Bread                             |
| +1     | Cheese       | Bread + Cheese                    |
| +2     | Tomato       | Bread + Cheese + Tomato           |
| +3     | Lettuce      | Bread + Cheese + Tomato + Lettuce |
| Result |              | <b>The Complete Sandwich!</b>     |

This is exactly how SQL JOINS build the result, step by step.

Now let us see this with REAL data from the Oracle HR database.

## The Query

```
SELECT first_name, last_name, salary, job_title,  
       department_name, city, country_name, region_name  
FROM hr.employees e  
     LEFT JOIN hr.departments d ON e.department_id = d.department_id  
     LEFT JOIN hr.jobs j ON e.job_id = j.job_id  
     LEFT JOIN hr.locations l ON d.location_id = l.location_id  
     LEFT JOIN hr.countries c ON l.country_id = c.country_id  
     LEFT JOIN hr.regions r ON c.region_id = r.region_id;
```

To understand this, we will use just 1 employee and show what happens after each JOIN. We keep the data tiny on purpose so you can see clearly.

### The Original Tables (just 1 row each)

hr.employees (alias: e)

| employee_id | first_name | job_id  | department_id | salary |
|-------------|------------|---------|---------------|--------|
| 107         | Alice      | IT_PROG | 60            | 5000   |

hr.departments (alias: d)

| department_id | department_name | location_id |
|---------------|-----------------|-------------|
| 60            | IT              | 1500        |

hr.jobs (alias: j)

| job_id  | job_title  |
|---------|------------|
| IT_PROG | Programmer |

hr.locations (alias: l)

| location_id | city   | country_id |
|-------------|--------|------------|
| 1500        | London | UK         |

hr.countries (alias: c)

| country_id | country_name   | region_id |
|------------|----------------|-----------|
| UK         | United Kingdom | 2         |

hr.regions (alias: r)

| region_id | region_name |
|-----------|-------------|
| 2         | Americas    |

# Step-by-Step: Watch the Result Grow

## Step 1: FROM employees

We start with just the employees table. This is our "bread" - the foundation.

| first_name | last_name | salary | job_id  | department_id |
|------------|-----------|--------|---------|---------------|
| Alice      | Smith     | 5000   | IT_PROG | 60            |

Result after Step 1: 5 columns, 1 row



LEFT JOIN departments d ON e.department\_id = d.department\_id

## Step 2: + departments

The database matches department\_id = 60. It finds "IT" and adds department\_name. Notice the result now has 6 columns - the old 5 plus the new one.

| first_name | last_name | salary | job_id  | department_id | department_name |
|------------|-----------|--------|---------|---------------|-----------------|
| Alice      | Smith     | 5000   | IT_PROG | 60            | IT              |

Result after Step 2: 6 columns. The new column is highlighted.



LEFT JOIN jobs j ON e.job\_id = j.job\_id

## Step 3: + jobs

Matches job\_id = IT\_PROG, finds "Programmer". Result grows to 7 columns. This JOIN works on the result from Step 2 (not just the original employees table).

| first_name | last_name | salary | job_id  | department_id | department_name | job_title  |
|------------|-----------|--------|---------|---------------|-----------------|------------|
| Alice      | Smith     | 5000   | IT_PROG | 60            | IT              | Programmer |

Result after Step 3: 7 columns.



LEFT JOIN locations l ON d.location\_id = l.location\_id

## Step 4: + locations (THIS IS THE KEY STEP!)

**Look at the ON clause: d.location\_id = l.location\_id**

Where does the database find "d.location\_id"? Does it go to the original departments table? NO! It looks at the result from Step 3. See the table above - it already has a "department\_id" column. The database read "location\_id" from the departments data that was merged back in Step 2. That data

is already sitting in the result.

Think of it this way: when you joined departments in Step 2, the database copied all departments columns (including location\_id) into the result. Now in Step 4, the database just reads location\_id from that result. It does NOT go back to the departments table.

| first_name | last_name | salary | job_id  | dept_id | dept_name | job_title  | city   |
|------------|-----------|--------|---------|---------|-----------|------------|--------|
| Alice      | Smith     | 5000   | IT_PROG | 60      | IT        | Programmer | London |

Result after Step 4: 8 columns. The new column "city" is highlighted.



LEFT JOIN countries c ON l.country\_id = c.country\_id

### Step 5: + countries

The database matches using "l.country\_id" - the country\_id from the locations data that was merged in Step 4. It finds "United Kingdom". Result grows to 9 columns.

| first_name | last_name | salary | job_id  | dept | job_title  | city   | country_name   |
|------------|-----------|--------|---------|------|------------|--------|----------------|
| Alice      | Smith     | 5000   | IT_PROG | IT   | Programmer | London | United Kingdom |

Result after Step 5: 9 columns.



LEFT JOIN regions r ON c.region\_id = r.region\_id

### Step 6: + regions

The database uses "c.region\_id" from the countries data merged in Step 5. Final result: 10 columns with data from all 6 tables!

| first_name | last_name | salary | job_id  | dept | job_title  | city   | country_name   | region_name |
|------------|-----------|--------|---------|------|------------|--------|----------------|-------------|
| Alice      | Smith     | 5000   | IT_PROG | IT   | Programmer | London | United Kingdom | Americas    |

Result after Step 6: 10 columns. COMPLETE!

---

## The Big Picture

| After Step | JOIN           | New Column Added | Total Columns |
|------------|----------------|------------------|---------------|
| Start      | FROM employees | -                | 5             |
| 1          | + departments  | department_name  | 6             |
| 2          | + jobs         | job_title        | 7             |
| 3          | + locations    | city             | 8             |
| 4          | + countries    | country_name     | 9             |
| 5          | + regions      | region_name      | 10            |

Each JOIN adds one new column. The result keeps growing. The database never goes back to the original tables - it always reads from the result that was already built.

## Answer to the Key Question

Question: In Step 4, where does d.location\_id come from?

Answer: It comes from the result that was already built in Steps 1-3. Back in Step 2, when we joined departments, ALL columns from departments (including location\_id) were copied into the result. In Step 4, the database simply reads location\_id from that result. It does NOT open the departments table again.

**THE GOLDEN RULE: Each JOIN works on the result of all previous JOINS combined. It does NOT go back to the original tables.**